

closedai-chatbot Modul I

Fejlesztési terv és működési specifikáció - egyetemi projekt tárgy

Feladat

Ez a dokumentum a `closedai-chatbot` projekt aktuális állapotához és tervezett bővítéseihez készített fejlesztési tervet írja le. A modul célja egy modellautókat forgalmazó webáruház chatbot asszisztensének backend oldali kiszolgálása. A chatbot természetes nyelvű kérdésekre válaszol, figyelembe tudja venni a beszélgetési előzményeket, és a termékekkel kapcsolatos kérdésekhez adatbázisból származó strukturált adatokat használhat.

A production felhasználói felület nem az ebben a repositoryban található `index.html`, hanem a DNN CMS rendszerbe ágyazott chatbot komponens. Az `index.html` csak egy kisméretű, fejlesztői tesztkliens, amellyel lokálisan ellenőrizhető a `POST /question` végpont működése.

A rendszer FastAPI alapú HTTP API-ból, OpenAI-kompatibilis LLM kliensből, Langfuse-integrációval megfigyelhető modellhívásokból, router és formatter promptokból, valamint SQL Serverhez kapcsolódó adatlekérdező rétegből áll.

Architektúra

A modul architektúrája öt, egymásra épülő rétegre bontható. A rétegek célja, hogy a DNN-be ágyazott production felület, a fejlesztői tesztkliens, az LLM-szolgáltató és az adatbázis-lekérdezések egymástól jól elkülöníthetők legyenek.

- DNN CMS frontend réteg: production chatbot UI, amely a backend API-t hívja.
- Fejlesztői tesztkliens: statikus `index.html`, kizárólag lokális kézi teszteléshez.
- HTTP API réteg: `src/main.py`, FastAPI alkalmazás request és response modellekkel.
- LLM orkestrációs réteg: `src/generate.py`, `src/llm/router.py`, `src/llm/formatter.py`.
- Adat-hozzáférési réteg: `src/data/sql_tools.py`, `src/database/config.py`, `src/database/engine.py`.

Rétegek feladatai

Réteg	Felelősség	Fő fájlok / rendszer
DNN CMS chatbot UI	Production felhasználói interakció és a chatbot beágyazása a webáruház CMS környezetébe.	DNN CMS komponens
Lokális tesztkliens	Kérdés beküldése, válasz és tool metaadat kézi ellenőrzése fejlesztés közben.	index.html
API	HTTP végpontok, Pydantic validáció, hibák egységes HTTP válasszá alakítása.	src/main.py
LLM vezérlés	Üzenetek összeállítása, router-modell futtatása, eszközhívások feloldása, végső válasz megfogalmazása.	src/generate.py , src/llm/*
SQL integráció	SQL Server kapcsolat létrehozása, termékadatok lekérdezése, JSON-kompatibilis átalakítás.	src/data/sql_tools.py , src/database/*
Konfiguráció és observability	LLM, Langfuse és adatbázis elérési paraméterek betöltése környezeti változókból.	.env , .env.sample

User Journal

Production környezetben a felhasználó a DNN CMS-be ágyazott chatbot felületen írja be a kérdését. A DNN komponens a kérdést, az opcionális beszélgetési előzményeket és a kívánt előzményablak méretét JSON formában elküldi a backend `POST /question` végpontjára.

A router-modell eldönti, hogy közvetlen válasz elég-e, vagy szükség van adatbázisos tool hívására. A modellautó-webáruházban tipikus kérdés lehet egy konkrét autó ára, készlete, hasonló alternatívák keresése, akciós termékek listázása, kategória szerinti böngészés vagy több modell összehasonlítása.

A válasz JSON-ként tér vissza. A DNN production felület ezt saját design rendszerének megfelelően jeleníti meg, míg a lokális tesztkliens egyszerű szöveggént és listaként rajzolja ki.

Felhasználói esetek

Eset	Leírás	Elvárt eredmény
Általános webáruházi kérdés	A felhasználó a modellautó-webáruházhoz kapcsolódó általános kérdést tesz fel.	A router nem hív SQL-eszközt, a modell közvetlen választ ad.
Népszerű termékek lekérdezése	A felhasználó top, népszerű vagy leggyakrabban foglalt termékekre kérdez.	A <code>get_hot_products</code> tool lefut, majd az eredmény természetes nyelvű válaszbá kerül.
Termékinformáció	A felhasználó egy adott modellautó árát, készletét, részleteit vagy hasonló termékeket kérdez.	A tervezett termék-toolok egyike ad strukturált adatot a formatter modellnek.
Beszélgetési folytatás	A felhasználó az előző válasza utal vissza.	A kliens átadja a legutóbbi előzményeket, a backend a beállított ablakméretig felhasználja őket.
Kézi fejlesztői ellenőrzés	A fejlesztő az <code>index.html</code> fájlból teszteli a végpontot.	A tesztkliens megjeleníti az API választ, de nem production UI.

Adatbázis terv

A jelenlegi modul a termékadatokat meglévő SQL Server adatbázisból olvassa. A kapcsolat ODBC-formátumú connection stringből készül, amelyet a `DB_CONNECTION_STRING` környezeti változó tartalmaz. A tervezett termék-toolok ugyanarra az adat-hozzáférési rétegre épülnek, de később több lekérdezést, szűrőt és összehasonlító műveletet vezetnek be.

Kapcsolat

Elem	Leírás
<code>DB_CONNECTION_STRING</code>	ODBC-stílusú SQL Server connection string a projektgyökérben lévő <code>.env</code> fájlban.
<code>get_database_url()</code>	A connection stringet URL-kódolja, és <code>mssql+pyodbc:///?odbc_connect=...</code> formára alakítja.
<code>get_engine()</code>	Singleton SQLAlchemy engine-t ad vissza <code>pool_pre_ping=True</code> beállítással.
<code>SessionLocal</code>	SQLAlchemy session factory, későbbi tranzakciós vagy repository-alapú fejlesztésekhez.

Termékadatok várható mezői

Mező	Típus	Leírás
<code>ProductId</code>	int/string	Termék azonosítója, toolok közötti hivatkozáshoz és összehasonlításhoz.
<code>ProductName</code>	string	A modellautó neve.
<code>Category</code>	string	Termékkategória, például versenyautó, klasszikus autó vagy kiegészítő.
<code>Scale</code>	string	Méretarány, például 1:18, 1:24 vagy 1:43.
<code>Brand</code>	string	Gyártó vagy márka.
<code>Price</code>	decimal	Aktuális ár, alapértelmezett pénznemként Ft kezeléssel.
<code>Stock</code>	int/string	Készletmennyiség vagy elérhetőségi állapot.
<code>Description</code>	string	Rövid vagy részletes termékleírás.

Szolgáltatások

A backend szolgáltatásai jelenleg függvényalapú modulokban találhatók. Logikailag külön szolgáltatásként kezelhető a chatbot orkestráció, az LLM-kliens, a router, a formatter, a tool resolver és az SQL tool réteg.

Chatbot orkestráció - generate_response

A `generate_response(question, history, history_window)` a chatbot központi belépési pontja. Feladata az LLM-kliens előkészítése, a modellnév ellenőrzése, az üzenetlista összeállítása, a router futtatása, az esetleges tool-hívás feloldása és a végső válasz visszaadása.

Paraméter	Típus	Leírás
<code>question</code>	string	Az aktuális felhasználói kérdés.
<code>history</code>	lista vagy null	Korábbi user és assistant üzenetek. Csak érvényes role és nem üres content kerül továbbításra.
<code>history_window</code>	int	Legfeljebb hány friss előzmény kerüljön be a modell kontextusába. API-szinten 0 és 20 közötti érték.

LLM kliens és Langfuse

A `get_llm()` a `.env` fájlból betölti az `API_KEY` és `BASE_URL` változókat, majd Langfuse OpenAI-kompatibilis klienst hoz létre. A tényleges modell nevét a `LLM_MODEL` adja meg. A Langfuse működéséhez a környezet tartalmazza a `LANGFUSE_SECRET_KEY`, `LANGFUSE_PUBLIC_KEY` és `LANGFUSE_BASE_URL` változókat is.

Router modell

A router a `ROUTER_SYSTEM_PROMPT` alapján a modellautó-webáruház kontextusában dönt. A router és formatter system promptjai a `src/prompts/system.py` fájlban vannak, nem a `.env`-ből érkeznek. A router feladata, hogy a felhasználói kérdéshez a legszűkebb szükséges toolt válassza ki, vagy tool nélkül adjon direkt választ.

Jelenlegi és tervezett tool-katalógus

Tool	Állapot	Feladat
<code>get_hot_products</code>	Meglévő alap tool	Népszerű, top vagy legtöbbször foglalt termékek lekérdezése.
<code>get_recommendation</code>	Előkészített bővítési pont	Termékajánlás későbbi szabályok és termékadatok alapján.
Termék-toolok	Tervezett	Részletes termékkeresés, ár, készlet, akciók, hasonló termékek és összehasonlítás kezelése.

Tervezett termék-toolok

A következő toolok a modellautókat forgalmazó webshop asszisztensi működését bővítik. A cél az, hogy a chatbot ne általános szöveg alapján válaszoljon termékadatokra, hanem ellenőrzött, strukturált adatot kérjen le, majd azt a formatter modell felhasználóbarát válasszá alakítsa.

Tool	Cél	Tervezett bemenet	Tervezett kimenet
<code>get_product_by_name</code>	Konkrét modellautó lekérdezése pontos vagy közel pontos terméknev alapján.	<code>name</code>	Termékazonosító, név, kategória, ár, készlet és rövid leírás.
<code>search_products</code>	Szabad szavas termékkeresés, például márka, típus, méretarány vagy kulcsszó alapján.	<code>query</code> , <code>limit</code>	Találati lista relevancia szerinti sorrendben.
<code>get_products_by_category</code>	Termékek listázása kategória szerint, például versenyautó, klasszikus autó, kamion vagy kiegészítő.	<code>category</code> , <code>limit</code>	Kategóriába tartozó termékek rövidített adatai.
<code>get_product_stock</code>	Készletinformáció lekérdezése egy adott modellautóhoz.	<code>product_id</code> vagy <code>name</code>	Aktuális készlet, elérhetőségi állapot, opcionálisan várható utánpótlás.
<code>get_product_price</code>	Aktuális ár lekérdezése egy adott termékhez.	<code>product_id</code> vagy <code>name</code>	Listaár, akciós ár, pénznem és árvényességi információ.
<code>get_discounted_products</code>	Akciós vagy kedvezményes modellautók listázása.	<code>limit</code> , <code>category</code>	Kedvezményes terméklista eredeti és akciós árakkal.
<code>get_products_by_price_range</code>	Termékek szűrése árintervallum alapján.	<code>min_price</code> , <code>max_price</code> , <code>category</code>	A megadott árhatárok közé eső termékek.
<code>get_product_details</code>	Részletes termékadatlap lekérése.	<code>product_id</code> vagy <code>name</code>	Leírás, méretarány, gyártó, anyag, elérhetőség, ár és kapcsolódó metaadatok.
<code>get_similar_products</code>	Hasonló modellautók ajánlása egy kiválasztott termék alapján.	<code>product_id</code> , <code>limit</code>	Hasonló kategóriájú, árú, márkájú vagy méretarányú termékek.
<code>compare_products</code>	Több modellautó összehasonlítása vásárlási döntés támogatásához.	<code>product_ids</code> vagy <code>names</code>	Összehasonlító adatok ár, készlet, kategória, méretarány és fő jellemzők szerint.

Formatter modell

A `run_formatter_model()` egy második LLM-hívással végleges, felhasználóbarát választ készít. Bemenete egy JSON objektum, amely tartalmazza a felhasználó kérdését, a route nevét, az SQL vagy tool eredményt és a direkt választ. A formatter prompt előírja, hogy a válasz maradjon a kapott adatokhoz kötve, ne találjon ki nem létező termékadatokat, és markdown nélkül, folyékony természetes nyelven fogalmazzon.

REST API

A backend FastAPI alkalmazásként fut. A CORS beállítás jelenleg fejlesztési célra teljesen nyitott: minden origin, metódus és header engedélyezett. Production DNN beágyazásnál ezt a tényleges CMS/domain originre kell korlátozni.

GET /

Visszaadja a projekt gyökerében található `index.html` fejlesztői tesztclient. Ez nem production felület, csak a backend gyors kézi ellenőrzésére szolgál.

GET /health

Egyszerű állapotellenőrző végpont. Sikeres működés esetén a válasz: `{ "status": "ok" }`.

POST /question

A chatbot fő végpontja. JSON kérést fogad, lefuttatja a válaszgenerálást, majd JSON választ ad vissza.

Kérés mező	Típus	Validáció	Leírás
<code>question</code>	string	1-4000 karakter	Az aktuális kérdés.
<code>history</code>	lista	Opcionális, alapérték üres lista	Korábbi üzenetek <code>user</code> vagy <code>assistant</code> role-lal.
<code>history_window</code>	int	0-20, alapérték 4	A figyelembe vett legutóbbi üzenetek száma.

Válasz mező	Típus	Leírás
<code>answer</code>	string	A felhasználónak megjelenítendő végső válasz.
<code>tool_used</code>	string vagy null	A használt eszköz neve, ha történt tool-hívás.
<code>data</code>	object vagy null	Strukturált kiegészítő adat, például terméklista, ár, készlet vagy összehasonlítási eredmény.

Tool eredmények tervezett szerkezete

A termék-toolok egységes, JSON-kompatibilis válaszokat adnak. A formatter modell minden esetben ebből az objektumból dolgozik. A production DNN felület dönthet úgy, hogy a `data` mezőből külön termékkártyákat vagy összehasonlító nézetet is rajzol.

Szerkezet	Használat
<code>{ "product": {...} }</code>	Egy konkrét termék részletei, ára vagy készlete.
<code>{ "products": [...] }</code>	Keresés, kategória, akciós termékek, árintervallum vagy hasonló termékek listája.
<code>{ "comparison": [...] }</code>	Több termék összehasonlításának strukturált eredménye.

Felhasználói felületek

DNN CMS-be ágyazott production chatbot

A production felület a DNN CMS részeként jelenik meg, ezért a végleges UI kialakítása, jogosultsági kontextusa és stílusa a CMS rendszerhez igazodik. A backend szempontjából a DNN komponens feladata a kérés JSON formátumú összeállítása, az API hívása, majd a válasz megjelenítése.

- A DNN komponens hívja a `POST /question` végpontot.
- A beszélgetési előzményeket a DNN oldali klienslogika adhatja át.
- A toolból érkező strukturált adatokat a CMS design rendszerének megfelelően kell kirajzolni.
- A production felületen külön kezelendő a jogosultság, naplózás, rate limit és felhasználói hibakommunikáció.

index.html fejlesztői teszt Kliens

A repository gyökerében található `index.html` egy önálló, statikus HTML fájl. Célja kizárólag az, hogy fejlesztés közben gyorsan ellenőrizhető legyen az API működése DNN környezet nélkül.

Prompt terv

A rendszer két külön, kódban tárolt system promptot használ. Ezek nem a `.env` konfiguráció részei, hanem a `src/prompts/system.py` fájlban találhatók.

Router prompt elvárásai

- A chatbot maradjon a modellautó-webáruház kontextusában.
- Csak akkor hívjon toolt, ha a kérdés egyértelműen illeszkedik egy elérhető vagy tervezett toolhoz.
- Konkrét termékadatoknál részesítse előnyben a strukturált adatot adó toolokat.
- Általános, információs vagy nem támogatott kérdésnél adjon direkt választ.

Formatter prompt elvárásai

- A kapott tool-eredményre támaszkodjon, ne találjon ki terméknevet, árat, készletet vagy kedvezményt.
- Az árakat forintként értelmezze, ha ármező érkezik.
- Legyen barátságos, természetes és vásárlási döntést segítő.
- Ne használjon markdown formázást, mert a jelenlegi tesztkliens sima szöveggént jeleníti meg a választ.

Konfiguráció

A projekt a gyökérben lévő `.env` fájlból olvassa a futásidejű beállításokat. A titkos értékeket nem szabad repositoryba commitolni. A repositoryban létrehozott `.env.sample` csak kulcsneveket és mintaértékeket tartalmaz.

Változó	Kötelező?	Leírás
API_KEY	Igen	Az OpenAI-kompatibilis LLM szolgáltató API kulcsa.
BASE_URL	Igen	Az LLM API alap URL-je, például OpenRouter vagy saját gateway.
LLM_MODEL	Igen	A használni kívánt modell azonosítója.
LANGFUSE_SECRET_KEY	Langfuse esetén igen	Langfuse secret kulcs a modellhívások megfigyeléséhez.
LANGFUSE_PUBLIC_KEY	Langfuse esetén igen	Langfuse public kulcs.
LANGFUSE_BASE_URL	Langfuse esetén igen	Langfuse szerver URL-je.
DB_CONNECTION_STRING	Igen SQL toolhoz	SQL Server ODBC connection string.

Biztonság és adatvédelem

A chatbot külső LLM szolgáltatóhoz továbbítja a felhasználói kérdéseket és a beszélgetési előzmények egy részét, ezért production bevezetésnél adatvédelmi és jogosultsági szabályokat kell alkalmazni.

- A `.env` fájlban lévő API kulcsok és adatbázis connection string titkos adatok.
- A teljesen nyitott CORS beállítást production DNN domainre kell korlátozni.
- A backend előtt jelenleg nincs saját bejelentkezés vagy jogosultságkezelés; productionben ezt a DNN környezettel össze kell hangolni.

- A felhasználói kérdések az LLM szolgáltatóhoz kerülnek, ezért személyes adatot csak szabályozott környezetben szabad beküldeni.
- A tervezett termék-tooloknál minden SQL-lekérdezést paraméterezetten kell implementálni.

Üzembehelyezés

Lokális fejlesztéshez a projekt telepíthető és futtatható az alábbi lépésekkel.

- Függőségek telepítése: `uv sync`.
- `.env` létrehozása a `.env.sample` alapján.
- API indítása: `uv run uvicorn main:app --app-dir src --reload --port 8080`.
- Lokális tesztkliens: `http://localhost:8080`.
- Health check: `http://localhost:8080/health`.
- Chat végpont: `POST http://localhost:8080/question`.

Production használatban a DNN CMS-be ágyazott frontend hívja ugyanezt az API-t. A `deploy.ps1` script friss kódot húz, szinkronizálja a függőségeket, majd újraindítja a `closedai-chatbot-api` Windows szolgáltatást.

Tesztelési terv

A repositoryban jelenleg nem található külön tesztcsomag, ezért a következő fejlesztési iterációban célzott unit és integrációs tesztet kell hozzáadni. A teszteknél az LLM-et és az adatbázist érdemes mockolni, hogy a futás ne függjön külső szolgáltatástól.

Tesztterület	Javasolt ellenőrzések
Üzenetépítés	<code>_build_messages</code> : history ablak, üres vagy hibás history elemek kihagyása.
Router resolver	Direkt válasz, meglévő tool, tervezett tool, ismeretlen tool és hibás argumentum kezelése.
Termék-toolok	Névkeresés, kategóriaszűrés, árintervallum, készlet, akciók, hasonló termékek és összehasonlítás tesztadatai.
SQL tool	Paraméterezett lekérdezések, Decimal és date/datetime JSON-kompatibilis átalakítása, üres eredményhalmaz kezelése.
API	<code>/health</code> válasz, <code>/question</code> request validáció, 500-as hibakezelés mockolt kivételekkel.
DNN production integráció	A DNN komponens által küldött request szerkezete, auth kontextus, CORS és felhasználói hibaüzenetek.

Kockázatok és korlátok

- Az LLM válaszai szolgáltatótól, modelltől és prompttól függenek, ezért nem teljesen determinisztikusak.
- A külső LLM szolgáltató kiesése vagy lassulása közvetlenül érinti a chatbot válaszidejét.
- A tervezett termék-tooloknál pontos adatmodellre és egységes tool-válasz szerkezetre lesz szükség.
- Nincs beépített rate limit, backend oldali felhasználói azonosítás vagy audit napló.
- Az API szinkron endpointként van megírva, miközben LLM- és adatbázishívásokat végez. Nagyobb terhelésnél aszinkron vagy háttérfeladatos kialakítás javasolt.

Fejlesztési ütemterv

Fázis	Tartalom	Eredmény
I. Alap chatbot backend	FastAPI endpoint, LLM-kliens, router és formatter folyamat, lokális tesztkliens.	A jelenlegi projekt működési alapja.
II. DNN production integráció	A DNN CMS chatbot komponens végleges API-használata, CORS és jogosultsági környezet pontosítása.	Beágyazott webáruházi chatbot felület.
III. Termék-tool katalógus	Keresés, kategória, ár, készlet, akció, részletek, hasonló termékek és összehasonlítás implementálása.	Adataalapú modellautó-asszisztens válaszok.
IV. Ajánlórendszer	<code>get_recommendation</code> szabályok, scoring és termékadat-alapú ajánlások kidolgozása.	Vásárlási döntést támogató ajánlások.
V. Élesítési keményítés	Auth integráció, CORS szűkítés, rate limit, naplózás, monitoring, tesztek és biztonságos hibaüzenetek.	Éles környezetben is védhető szolgáltatás.

Fájlstruktúra összefoglaló

Fájl	Szerep
src/main.py	FastAPI alkalmazás, request/response modellek, HTTP végpontok.
src/generate.py	A chatbot válaszgenerálásának fő folyamata.
src/llm/client.py	LLM-kliens létrehozása környezeti változókból.
src/llm/router.py	Tool deklarációk és router-modell hívása.
src/llm/formatter.py	SQL vagy direkt válasz végső természetes nyelvi megfogalmazása.
src/data/router_tools.py	Router döntésének Python toolhívássá alakítása.
src/data/sql_tools.py	SQL lekérdezések és JSON-kompatibilis eredménykonverzió.
src/database/config.py	Adatbázis URL előállítás.
src/database/engine.py	SQLAlchemy engine és session factory.
src/prompts/system.py	Router és formatter system promptok.
index.html	Fejlesztői, lokális tesztkliens, nem production UI.
.env.sample	Titokmentes konfigurációs minta a valós .env kulcsai alapján.

Összegzés

A closedai-chatbot projekt egy modellautó-webáruház chatbot backend alapját adja. A végleges felhasználói élmény a DNN CMS-be ágyazott chatbot komponensen keresztül jelenik meg, miközben a repositoryban található index.html csak fejlesztői tesztkliens.

A következő nagy bővítési irány a termék-tool katalógus kialakítása. A tervezett toolokkal a chatbot képes lesz konkrét modellautókat keresni, kategóriák és ár alapján szűrni, készletet és árat ellenőrizni, akciós termékeket listázni, hasonló modelleket ajánlani és több terméket összehasonlítani. Ezzel a rendszer általános chatbotból adatalapú, vásárlási döntést segítő asszisztenssé válik.